



5

7

840 sp

عملي

كلية الهندسة المعلوماتية

السنة الثالثة

Vision and Control



RB Informatics; 10/11/2024

عمل طلابي

محتوى مجاني غير مخصص للبيع التجاري

بيانات حاسوبية

مصفوفة الأزرار (keys Array)

1. ما هي مصفوفة الأزرار؟

مصفوفة الأزرار هي مصفوفة تستخدم لتتبع حالة الأزرار الموجودة على لوحة المفاتيح في تطبيقات OpenGL. تم تعريف المصفوفة في الكود كالتالي:

```
bool keys[256]; // Array Used For The Keyboard Routine
```

حجم المصفوفة: تحتوي على 256 عنصراً، مما يتيح لها تتبع حالة كل زر ممكن على لوحة المفاتيح (من 0 إلى 255).
القيمة: تخزن كل قيمة في المصفوفة كـ true (مضغوط) أو false (غير مضغوط).

2. استخدام المصفوفة في DrawGLScene:

تستخدم مصفوفة الأزرار في الدالة DrawGLScene للتحكم في الحركة أو الرؤية بناءً على الأزرار التي يضغط عليها المستخدم.

مثال على التحكم في الحركة:

يمكننا تعديل قيم المتغيرات المسؤولة عن الحركة باستخدام المصفوفة keys.

Push harder than yesterday if you
want a different tomorrow



```

float PosX = 0.0f;
float PosZ = 0.0f;

void DrawGLScene(GLvoid) {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();
    glTranslated(PosX, 0, PosZ);
    if (keys['W']) {
        PosZ -= 0.1f;
    }
    if (keys['S']) {
        PosZ += 0.1f;
    }
    if (keys['A']) {
        PosX -= 0.1f;
    }
    if (keys['D']) {
        PosX += 0.1f;
    }
    glBegin(GL_QUADS);
    glVertex3f(-1.0f, -1.0f, -10.0f);
    glVertex3f(1.0f, -1.0f, -10.0f);
    glVertex3f(1.0f, 1.0f, -10.0f);
    glVertex3f(-1.0f, 1.0f, -10.0f);
    glEnd();
    glFlush();
    SwapBuffers(hDC);
}

```

ملاحظات:

- يتم التعبير عن المحارف ب Capital Letter أي نكتب A وليس a.
- يمكن أن نستخدم باقي الأزرار مثل الأسهم (أعلى keys[VK_UP] أو أسفل keys[VK_DOWN] أو ...).
- يمكننا أن نضع الأكود التي نتعامل فيها مع مصفوفة الـ keys في تابع قبل DrawGLScene ثم استدعائه داخله وذلك لجعل الكود أوضح.



Mouse

ستحتاج إلى تنفيذ خطوات محددة للتقاط حركة الماوس والتفاعل مع نقراته داخل نافذة التطبيق:

1. إعداد حالات الماوس في الدالة WndProc:

الدالة WndProc تستخدم لمعالجة جميع الرسائل التي يرسلها نظام التشغيل أو النوافذ الأخرى إلى نافذة التطبيق. بالإضافة التحكم بالماوس، نضيف عدداً من الحالات (cases) التي تتعامل مع أحداث الماوس المختلفة داخل التابع LRESULT CALLBACK WndProc الموجود في التيمبلت.

```
case WM_MOUSEMOVE: {
    mouseX= (int)LOWORD(lParam); mouseY= (int)HIWORD(lParam);
    isClicked = (LOWORD(wParam) & MK_LBUTTON) ? true : false;
    isRClicked= (LOWORD(wParam) & MK_RBUTTON) ? true : false;
    break;
}
case WM_LBUTTONDOWN:
    isClicked = true; break;
case WM_RBUTTONDOWN:
    isRClicked= true; break;
```

WM_MOUSEMOVE: تلتقط إحداثيات الماوس عند تحريكه.

mouseY و mouseX تخزن موقع الماوس بناءً على إحداثيات الشاشة (والتي تختلف عن إحداثيات OpenGL).

isRClicked و isClicked تستخدم للتحقق من حالة أزرار الماوس (إذا كان الزر الأيسر أو الأيمن مضغوطاً).

WM_LBUTTONDOWN و WM_RBUTTONDOWN: تعيد القيمة false إلى المتغيرات isClicked (للزر الأيسر) و

isRClicked (للزر الأيمن) عندما تتغير حالة زر الماوس ليصبح غير مضغوط.

WM_LBUTTONDOWN و WM_RBUTTONDOWN: تعطي المتغيرات isClicked و isRClicked قيم true

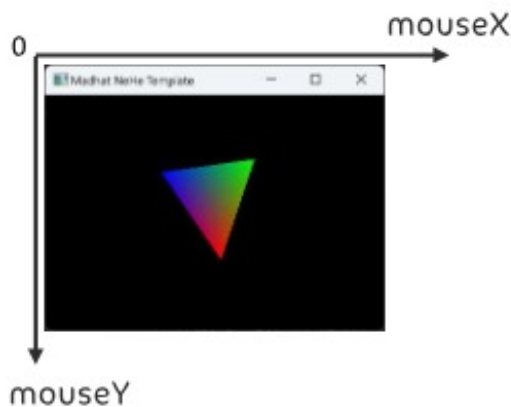
عندما تتغير حالة زر الماوس ليصبح مضغوط.

2. تعريف المتغيرات اللازمة

نعرف المتغيرات التي سنستخدمها لحفظ إحداثيات الماوس وحالة الأزرار بالشكل التالي وقبل تابع DrawGLScene لنتمكن من استخدامها به:

```
int mouseX = 0;
int mouseY = 0;
bool isClicked = false;
bool isRClicked = false;
```

3. نطاق القيم للمتغيرات:



mouseX: يأخذ قيمًا من 0 إلى عرض الشاشة (Screen Width).

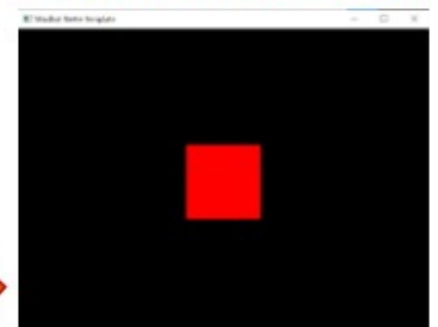
mouseY: يأخذ قيمًا من 0 إلى ارتفاع الشاشة (Screen Height).

isClicked و isRClicked: يأخذان قيمًا true أو false لتحديد حالة النقر.

4. استخدام أزرار الماوس:

مثال:

```
glTranslatef(0.0f, 0.0f, -10.0f); // Set the position of the shape
if (isClicked) {
    // Draw a cube when the left
    // mouse button is pressed
    glColor3f(1.0f, 0.0f, 0.0f);
    // Red color
    glBegin(GL_QUADS);
    // Front face of the cube
    glVertex3f(-1.0f, -1.0f, 1.0f);
    glVertex3f( 1.0f, -1.0f, 1.0f);
    glVertex3f( 1.0f, 1.0f, 1.0f);
    glVertex3f(-1.0f, 1.0f, 1.0f);
    // Other faces of the
    // cube can be added here...
    glEnd();
} else if (isRClicked) {
    // Draw a sphere when
    // the right mouse button is pressed
    glColor3f(0.0f, 0.5f, 1.0f); // Blue
    // step 1:
    GLUquadric* quad = gluNewQuadric();
    // step 2:
    gluSphere(quad, 1.5, 32, 32);
    // step 3:
    gluDeleteQuadric(quad);
}
```



Step1:

إنشاء GLUQuadric object يمكن استخدامه لرسم أشكال ثلاثية الأبعاد (quadric objects) مثل كرة أو أسطوانة أو مخروط.

Step2:

إنشاء كرة باستخدام الـ object (quad):

يأخذ التابع glusphere المعاملات التالية:

Object من نوع GLUQuadric وهو الذي يحتوي إعدادات الرسم.

radius نصف القطر

عدد الشرائح الطويلة slices أي عدد التقسيمات حول المحور z.

عدد الشرائح العرضية stacks أي عدد التقسيمات على طول المحور z.

فينتج لدينا كرة ملساء ذات دقة عالية.

Step3:

حذف الـ object بعد استخدامه لتجنب الـ memory leaks.

■ ملاحظة: يبدو الشكل في شاشة الخرج على شكل دائرة ولكنه في الحقيقة كرة ولها حجم، وسنتعلم لاحقاً كيف نضفي طابعاً ثلاثي الأبعاد عليها من خلال الإضاءة.

Viewing Transformation

نستخدم لتحديد موقع المراقب أو الكاميرا في مشهد OpenGL، مما يسمح لنا بتغيير وجهة النظر، وتشبه إلى حد كبير وضع كاميرا وتوجيهها نحو الأجسام المراد مشاهدتها.

استخدام الدالة gluLookAt:

لتحقيق ذلك، يمكننا استخدام الدالة التالية:

```
void gluLookAt(GLdouble eyeX, GLdouble eyeY, GLdouble eyeZ,
               GLdouble centerX, GLdouble centerY, GLdouble centerZ,
               GLdouble upX, GLdouble upY, GLdouble upZ);
```

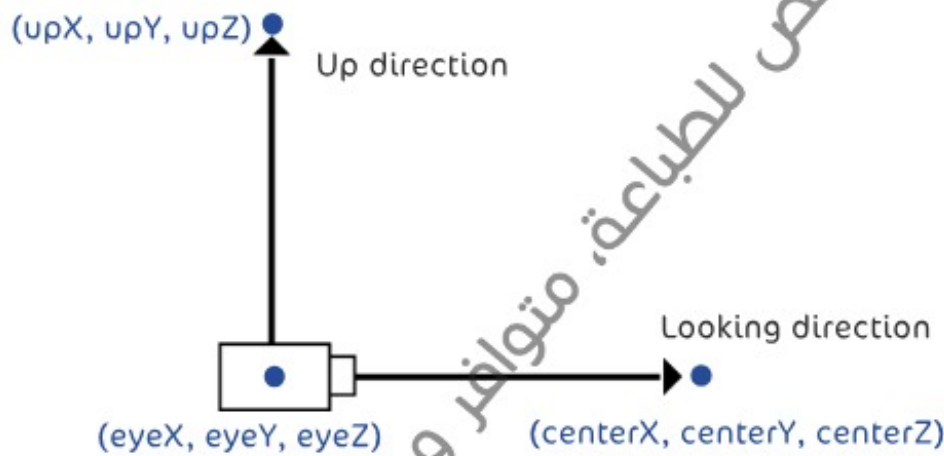


حيث أن:

$eyeX, eyeY, eyeZ$: تحدد موقع الكاميرا في المشهد، ويمثل هذا الموقع النقطة التي يقف فيها المراقب (أو الكاميرا) ويراقب المشهد منها. على سبيل المثال، إذا كانت الكاميرا في النقطة $(0, 0, 5)$ ، فهي تقف على المحور Z الموجب، على بُعد 5 وحدات من مركز المشهد.

$centerX, centerY, centerZ$: تحدد النقطة التي تنظر إليها الكاميرا. والتي تعرف باتجاه النظر. وبشكل افتراضي تكون هذه النقطة هي $(0, 0, -1)$ ، مما يعني أن الكاميرا تنظر نحو المحور Z السالب.

upX, upY, upZ : تحدد الاتجاه الذي يُعتبر عمودياً بالنسبة للكاميرا. يعرف هذا الاتجاه بـ (UP direction). الافتراضي له هو $(0, 1, 0)$ ، مما يعني أن المحور Y الموجب هو الاتجاه العمودي على الكاميرا.



مثال:

إذا كنا نريد وضع الكاميرا في النقطة $(0, 0, 5)$ وأن ننظر إلى مركز المشهد عند النقطة $(0, 0, 0)$ ونريد أن يكون المحور Y direction

`gluLookAt(0, 0, 5, // موقع الكاميرا`

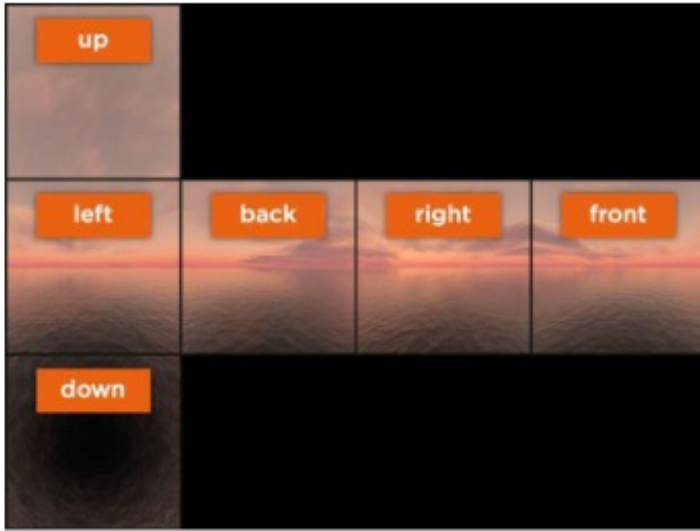
`0, 0, 0, // النقطة التي تنظر إليها الكاميرا`

`0, 1, 0); // الاتجاه الرأسي`

بهذا الشكل، ستظل الكاميرا تركز على مركز المشهد من النقطة $(0, 0, 5)$ ، مع الحفاظ على الاتجاه الرأسي هو المحور Y في النهاية يتم توحيد المفاهيم الثلاثة للحصول على تابع الكاميرا المناسب والمريح للحركة.



skybox



نستخدم skybox لخلق بيئة واقعية بشكل واسع. نحتاج إلى مجموعة من 6 صور تتوافق مع الجوانب الستة للمكعب.

حيث نعتبر أنفسنا داخل مكعب ونقوم بلصق الصور عليه من الداخل ليعطينا مثل وهم بماهية المكان حولنا بدلاً من السواد الذي نراه في عالم الـ OpenGL.

سنوضح بعض الأفكار: الـ skybox، محاكاة حركة الإنسان عند المشي وذلك باستخدام تابع الـ sin في تابع الكاميرا، استخدام إحداثيات الماوس في التحريك، وذلك عن طريق الكود الموجود على رابط الدرايف التالي:



ملاحظة: يوجد نسختين من المشروع، نسخة للـ clion ونسخة للـ visual studio.

Don't compare yourself to others.
Be like the sun and the moon and
shine when it's your time

